

ACTIONS ASYNCHRONES

Cours Theorique

Session 17 : createAsyncThunk & API

Gerer les appels API avec Redux Toolkit

Objectifs de la seance :

- Comprendre la **problematique** des appels API dans Redux
- Maitriser **createAsyncThunk** pour les actions asynchrones
- Gerer les **3 etats** : pending, fulfilled, rejected
- Utiliser **extraReducers** pour reagir aux actions async
- Implementer la gestion **loading/error** dans l'UI

Duree : 5h
Prerequis : S14-S16 (Redux, RTK, Selectors)
Prochaine session : S18 - Formulaires & Listes

Table des matières

1	Problematique des Appels API	2
1.1	Le Probleme	2
1.2	Exemple du Probleme	2
1.3	La Solution : Middleware Thunk	2
2	Thunk sans Redux Toolkit (Methode Classique)	3
2.1	Installation	3
2.2	Configuration du Store avec Thunk	3
2.3	Types d'Actions	3
2.4	Action Creators Synchrones	3
2.5	Thunk : Action Creator Asynchrone	5
2.6	Reducer Classique	6
2.7	Utilisation dans un Composant	6
2.8	Comparaison : Classique vs Redux Toolkit	8
3	createAsyncThunk (Redux Toolkit)	9
3.1	Definition	9
3.2	Syntaxe de Base	9
3.3	Les 3 Actions Generees	9
3.4	Thunk avec Parametre	10
3.5	Thunk avec Gestion d'Erreur	10
4	extraReducers : Reagir aux Actions Async	11
4.1	Pourquoi extraReducers?	11
4.2	Syntaxe avec Builder	12
4.3	Explication du State	13
4.4	Pattern Complet : State Structure	13
5	Utilisation dans les Composants	13
5.1	Dispatch un Thunk	13
5.2	Affichage des Etats : Loading et Error	14
6	Exemple Complet : Users CRUD	15
6.1	Structure du Projet	15
6.2	usersSlice.js Complet	15
7	API de Test : JSONPlaceholder	18
7.1	Presentation	18
7.2	Endpoints Disponibles	18
7.3	Exemple de Reponse	18
8	Recapitulatif	19
8.1	Workflow Complet	19
8.2	Tableau Recapitulatif	19
8.3	Checklist Session 17	19

1 Problematique des Appels API

1.1 Le Probleme

Dans une application React, on a souvent besoin de :

- **Recuperer** des donnees depuis un serveur (GET)
- **Envoyer** des donnees au serveur (POST, PUT, DELETE)
- **Gerer** les etats de chargement et d'erreur

Attention

Probleme : Les reducers Redux sont des fonctions **synchrones et pures**. Ils ne peuvent pas :

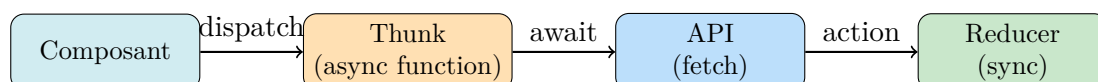
- Faire des appels `fetch()` ou `axios`
- Utiliser `async/await`
- Avoir des effets de bord (side effects)

1.2 Exemple du Probleme

```
1 // INTERDIT dans un reducer !
2 const usersSlice = createSlice({
3   name: 'users',
4   initialState: { items: [] },
5   reducers: {
6     // ERREUR : Un reducer ne peut pas etre async
7     fetchUsers: async (state) => {
8       const response = await fetch('/api/users'); // INTERDIT !
9       state.items = await response.json();
10    }
11  }
12 });
```

1.3 La Solution : Middleware Thunk

Redux Toolkit inclut automatiquement le middleware **redux-thunk** qui permet de dispatcher des **fonctions** (et pas seulement des objets action).



2 Thunk sans Redux Toolkit (Methode Classique)

Avant Redux Toolkit, on utilisait le middleware `redux-thunk` manuellement. Cette section montre la methode **classique** pour comprendre ce que RTK simplifie.

2.1 Installation

```
# Methode classique (SANS RTK)
npm install redux react-redux redux-thunk
```

2.2 Configuration du Store avec Thunk

```
1 // store.js - Methode CLASSIQUE (sans RTK)
2 import { createStore, applyMiddleware, combineReducers } from 'redux';
3 import thunk from 'redux-thunk';
4 import usersReducer from './reducers/usersReducer';
5
6 const rootReducer = combineReducers({
7   users: usersReducer
8 });
9
10 // Appliquer le middleware thunk manuellement
11 const store = createStore(
12   rootReducer,
13   applyMiddleware(thunk)
14 );
15
16 export default store;
```

2.3 Types d'Actions

```
1 // actions/usersTypes.js
2 export const FETCH_USERS_REQUEST = 'FETCH_USERS_REQUEST'; // Pending
3 export const FETCH_USERS_SUCCESS = 'FETCH_USERS_SUCCESS'; // Fulfilled
4 export const FETCH_USERS_FAILURE = 'FETCH_USERS_FAILURE'; // Rejected
```

2.4 Action Creators Synchrones

```
1 // actions/usersActions.js
2 import { FETCH_USERS_REQUEST, FETCH_USERS_SUCCESS, FETCH_USERS_FAILURE }
3   from './usersTypes';
4
5 // Action pour demarrer le chargement
6 export const fetchUsersRequest = () => ({
7   type: FETCH_USERS_REQUEST
8 });
9
10 // Action pour le succes (avec les donnees)
11 export const fetchUsersSuccess = (users) => ({
12   type: FETCH_USERS_SUCCESS,
13   payload: users
14 });
15
16 // Action pour l'echec (avec l'erreur)
```

```
17 export const fetchUsersFailure = (error) => ({  
18   type: FETCH_USERS_FAILURE,  
19   payload: error  
20 });
```

2.5 Thunk : Action Creator Asynchrone

Syntaxe

```
1 // actions/usersActions.js
2
3 // THUNK : Une fonction qui retourne une fonction async
4 // Cette fonction recoit dispatch en parametre
5 export const fetchUsers = () => {
6   return async (dispatch) => {
7     // 1. Dispatcher l'action PENDING
8     dispatch(fetchUsersRequest());
9
10    try {
11      // 2. Faire l'appel API
12      const response = await fetch(
13        'https://jsonplaceholder.typicode.com/users'
14      );
15
16      if (!response.ok) {
17        throw new Error('Erreur serveur');
18      }
19
20      const data = await response.json();
21
22      // 3. Dispatcher l'action SUCCESS avec les donnees
23      dispatch(fetchUsersSuccess(data));
24
25    } catch (error) {
26      // 4. Dispatcher l'action FAILURE avec l'erreur
27      dispatch(fetchUsersFailure(error.message));
28    }
29  };
30 };
```

Important - A retenir pour l'examen

Un **Thunk** est une fonction qui :

1. Retourne une **autre fonction** (au lieu d'un objet action)
2. Cette fonction recoit **dispatch** (et optionnellement **getState**)
3. Peut faire des operations **asynchrones** (fetch, timeout, etc.)
4. Dispatch des actions **synchrones** au bon moment

2.6 Reducer Classique

```
1 // reducers/usersReducer.js
2 import { FETCH_USERS_REQUEST, FETCH_USERS_SUCCESS, FETCH_USERS_FAILURE }
3   from '../actions/usersTypes';
4
5 const initialState = {
6   items: [],
7   loading: false,
8   error: null
9 };
10
11 const usersReducer = (state = initialState, action) => {
12   switch (action.type) {
13     case FETCH_USERS_REQUEST:
14       return {
15         ...state,
16         loading: true,
17         error: null
18       };
19
20     case FETCH_USERS_SUCCESS:
21       return {
22         ...state,
23         loading: false,
24         items: action.payload
25       };
26
27     case FETCH_USERS_FAILURE:
28       return {
29         ...state,
30         loading: false,
31         error: action.payload
32       };
33
34     default:
35       return state;
36   }
37 };
38
39 export default usersReducer;
```

2.7 Utilisation dans un Composant

```
1 // components/UsersList.jsx
2 import { useEffect } from 'react';
3 import { useDispatch, useSelector } from 'react-redux';
4 import { fetchUsers } from '../actions/usersActions';
5
6 function UsersList() {
7   const dispatch = useDispatch();
8   const { items, loading, error } = useSelector(state => state.users);
9
10  useEffect(() => {
11    dispatch(fetchUsers()); // Dispatcher le thunk
12  }, [dispatch]);
13}
```

```
14   if (loading) return <p>Chargement...</p>;  
15   if (error) return <p>Erreur : {error}</p>;  
16  
17   return (  
18     <ul>  
19       {items.map(user => <li key={user.id}>{user.name}</li>)}  
20     </ul>  
21   );  
22 }
```


2.8 Comparaison : Classique vs Redux Toolkit

asynorange!20 Methode Clas- sique (redux-thunk)	Redux Toolkit (createAsync-Thunk)
Installation	
redux redux-thunk react-redux	@reduxjs/toolkit react-redux
Configuration Store	
applyMiddleware(thunk) manuel	Thunk inclus automatiquement
Types d'actions	
3 constantes a definir manuellement	Generes automatiquement
Action Creators	
3 fonctions + 1 thunk a ecrire	1 seul createAsyncThunk
Reducer	
switch/case avec spread operator	extraReducers avec Immer
Nombre de lignes (environ)	
60 lignes	25 lignes

Attention

En pratique : Utilisez toujours `createAsyncThunk` de RTK dans vos projets. La methode classique est presentee ici uniquement pour **comprendre le fonctionnement interne** des thunks.

3 createAsyncThunk (Redux Toolkit)

3.1 Definition

Definition

`createAsyncThunk` est une fonction de Redux Toolkit qui genere automatiquement :

- Un **thunk** (fonction async dispatchable)
- **3 action creators** : pending, fulfilled, rejected
- La gestion automatique du **cycle de vie** de la requete

3.2 Syntaxe de Base

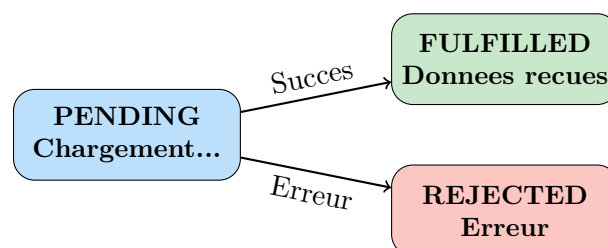
Syntaxe

```
1 import { createAsyncThunk } from '@reduxjs/toolkit';
2
3 // Creer un thunk pour fetcher les utilisateurs
4 export const fetchUsers = createAsyncThunk(
5   'users/fetchUsers',      // Nom de l'action (type prefix)
6   async () => {            // Fonction async (payload creator)
7     const response = await fetch('https://jsonplaceholder.typicode.
8       com/users');
9     const data = await response.json();
10    return data;             // Devient action.payload si succes
11  }
12 );
```

3.3 Les 3 Actions Generees

`createAsyncThunk` genere automatiquement 3 types d'actions :

asynorange!20 Action	Type genere	Quand ?
pending	users/fetchUsers/pending	Requete en cours
fulfilled	users/fetchUsers/fulfilled	Requete reussie (avec data)
rejected	users/fetchUsers/rejected	Requete echouee (avec error)



3.4 Thunk avec Parametre

```
1 // Thunk qui accepte un parametre (id)
2 export const fetchUserById = createAsyncThunk(
3   'users/fetchUserById',
4   async (userId) => { // userId est le parametre passe au dispatch
5     const response = await fetch(
6       `https://jsonplaceholder.typicode.com/users/${userId}`
7     );
8     const data = await response.json();
9     return data;
10  }
11 );
12
13 // Utilisation dans un composant
14 dispatch(fetchUserById(5)); // Fetch l'utilisateur avec id=5
```

3.5 Thunk avec Gestion d'Erreur

```
1 export const fetchUsers = createAsyncThunk(
2   'users/fetchUsers',
3   async (_, { rejectWithValue }) => {
4     try {
5       const response = await fetch('https://api.example.com/users');
6
7       // Verifier si la reponse est OK
8       if (!response.ok) {
9         throw new Error('Erreur serveur');
10      }
11
12      const data = await response.json();
13      return data;
14    } catch (error) {
15      // Retourner l'erreur de maniere controlee
16      return rejectWithValue(error.message);
17    }
18  }
19 );
20 );
```

Important - A retenir pour l'examen

rejectWithValue permet de personnaliser le payload de l'action **rejected**. Sans lui, l'erreur serait serialisee automatiquement (ce qui peut causer des problemes).

4 extraReducers : Reagir aux Actions Async

4.1 Pourquoi extraReducers ?

Les actions generees par `createAsyncThunk` ne sont **pas definies** dans le slice. On utilise `extraReducers` pour y reagir.

Definition

`extraReducers` permet a un slice de reagir a des actions **definies ailleurs** (thunks, autres slices, etc.).

4.2 Syntaxe avec Builder

Syntaxe

```
1 import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
2
3 // 1. Creer le thunk
4 export const fetchUsers = createAsyncThunk(
5   'users/fetchUsers',
6   async () => {
7     const response = await fetch('https://jsonplaceholder.typicode.com/users');
8     return response.json();
9   }
10 );
11
12 // 2. Creer le slice avec extraReducers
13 const usersSlice = createSlice({
14   name: 'users',
15   initialState: {
16     items: [],
17     loading: false,
18     error: null
19   },
20   reducers: {
21     // Actions synchrones ici (si besoin)
22   },
23   extraReducers: (builder) => {
24     builder
25       // Cas PENDING : requete en cours
26       .addCase(fetchUsers.pending, (state) => {
27         state.loading = true;
28         state.error = null;
29       })
30       // Cas FULFILLED : requete reussie
31       .addCase(fetchUsers.fulfilled, (state, action) => {
32         state.loading = false;
33         state.items = action.payload;
34       })
35       // Cas REJECTED : requete echouee
36       .addCase(fetchUsers.rejected, (state, action) => {
37         state.loading = false;
38         state.error = action.payload || action.error.message;
39       });
40   }
41 });
```

4.3 Explication du State

asynorange!20 Etat	loading	items	error
Initial	false	[]	null
Pending	true	[]	null
Fulfilled	false	[...data]	null
Rejected	false	[]	"message"

4.4 Pattern Complet : State Structure

Bonne Pratique

Structure recommandée pour un slice avec données async :

```
const initialState = {
  items: [],          // Les données
  loading: false,     // Etat de chargement
  error: null         // Message d'erreur (ou null)
};
```

5 Utilisation dans les Composants

5.1 Dispatcher un Thunk

```
1 import { useEffect } from 'react';
2 import { useDispatch, useSelector } from 'react-redux';
3 import { fetchUsers } from '../features/users/usersSlice';
4
5 function UsersList() {
6   const dispatch = useDispatch();
7
8   // Selectors pour lire le state
9   const users = useSelector((state) => state.users.items);
10  const loading = useSelector((state) => state.users.loading);
11  const error = useSelector((state) => state.users.error);
12
13  // Dispatcher le thunk au montage du composant
14  useEffect(() => {
15    dispatch(fetchUsers());
16  }, [dispatch]);
17
18  // Affichage conditionnel
19  if (loading) return <p>Chargement...</p>;
20  if (error) return <p>Erreur : {error}</p>;
21
22  return (
23    <ul>
24      {users.map(user => (
25        <li key={user.id}>{user.name}</li>
26      ))}
27    </ul>
28  );
29 }
```

5.2 Affichage des Etats : Loading et Error

```
1 function UsersList() {
2   const dispatch = useDispatch();
3   const { items, loading, error } = useSelector((state) => state.users);
4
5   useEffect(() => {
6     dispatch(fetchUsers());
7   }, [dispatch]);
8
9   // Etat PENDING
10  if (loading) {
11    return (
12      <div className="loading">
13        <span className="spinner"></span>
14        <p>Chargement des utilisateurs...</p>
15      </div>
16    );
17  }
18
19  // Etat REJECTED
20  if (error) {
21    return (
22      <div className="error">
23        <p>Erreur : {error}</p>
24        <button onClick={() => dispatch(fetchUsers())}>
25          Reessayer
26        </button>
27      </div>
28    );
29  }
30
31  // Etat FULFILLED
32  return (
33    <div className="users-list">
34      <h2>Utilisateurs ({items.length})</h2>
35      <ul>
36        {items.map(user => (
37          <li key={user.id}>
38            {user.name} - {user.email}
39          </li>
40        ))}
41      </ul>
42    </div>
43  );
44 }
```

6 Exemple Complet : Users CRUD

6.1 Structure du Projet

```
src/  
  app/  
    store.js  
  features/  
    users/  
      usersSlice.js      # Slice + Thunks  
      usersSelectors.js  # Selectors  
  components/  
    UsersList.jsx  
    UserCard.jsx  
  App.jsx  
  main.jsx
```

6.2 usersSlice.js Complet

```
1  import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';  
2  
3  const API_URL = 'https://jsonplaceholder.typicode.com/users';  
4  
5  // ===== THUNKS =====  
6  
7  // GET : Recuperer tous les users  
8  export const fetchUsers = createAsyncThunk(  
9    'users/fetchUsers',  
10   async (_, { rejectWithValue }) => {  
11     try {  
12       const response = await fetch(API_URL);  
13       if (!response.ok) throw new Error('Erreur serveur');  
14       return response.json();  
15     } catch (error) {  
16       return rejectWithValue(error.message);  
17     }  
18   }  
19 );  
20  
21 // GET : Recuperer un user par ID  
22 export const fetchUserById = createAsyncThunk(  
23   'users/fetchUserById',  
24   async (userId, { rejectWithValue }) => {  
25     try {  
26       const response = await fetch(`${API_URL}/${userId}`);  
27       if (!response.ok) throw new Error('Utilisateur non trouve');  
28       return response.json();  
29     } catch (error) {  
30       return rejectWithValue(error.message);  
31     }  
32   }  
33 );  
34  
35 // POST : Ajouter un user  
36 export const addUser = createAsyncThunk(  
37   'users/addUser',
```



```
38   async (userData, { rejectWithValue }) => {
39     try {
40       const response = await fetch(API_URL, {
41         method: 'POST',
42         headers: { 'Content-Type': 'application/json' },
43         body: JSON.stringify(userData)
44       });
45       if (!response.ok) throw new Error('Erreur creation');
46       return response.json();
47     } catch (error) {
48       return rejectWithValue(error.message);
49     }
50   }
51 );
52
53 // DELETE : Supprimer un user
54 export const deleteUser = createAsyncThunk(
55   'users/deleteUser',
56   async (userId, { rejectWithValue }) => {
57     try {
58       const response = await fetch(`${API_URL}/${userId}`, {
59         method: 'DELETE'
60       });
61       if (!response.ok) throw new Error('Erreur suppression');
62       return userId; // Retourner l'ID pour le supprimer du state
63     } catch (error) {
64       return rejectWithValue(error.message);
65     }
66   }
67 );
```

```
1 // ===== SLICE =====
2
3 const usersSlice = createSlice({
4   name: 'users',
5   initialState: {
6     items: [],
7     selectedUser: null,
8     loading: false,
9     error: null
10  },
11  reducers: {
12    // Actions synchrones
13    clearError: (state) => {
14      state.error = null;
15    },
16    clearSelectedUser: (state) => {
17      state.selectedUser = null;
18    }
19  },
20  extraReducers: (builder) => {
21    builder
22      // === FETCH ALL USERS ===
23      .addCase(fetchUsers.pending, (state) => {
24        state.loading = true;
25        state.error = null;
26      })
27      .addCase(fetchUsers.fulfilled, (state, action) => {
28        state.loading = false;
29        state.items = action.payload;
30      })
31      .addCase(fetchUsers.rejected, (state, action) => {
32        state.loading = false;
33        state.error = action.payload;
34      })
35
36      // === FETCH USER BY ID ===
37      .addCase(fetchUserById.fulfilled, (state, action) => {
38        state.selectedUser = action.payload;
39      })
40
41      // === ADD USER ===
42      .addCase(addUser.fulfilled, (state, action) => {
43        state.items.push(action.payload);
44      })
45
46      // === DELETE USER ===
47      .addCase(deleteUser.fulfilled, (state, action) => {
48        state.items = state.items.filter(user => user.id !== action.
49          payload);
50      });
51  });
52
53 export const { clearError, clearSelectedUser } = usersSlice.actions;
54 export default usersSlice.reducer;
```

7 API de Test : JSONPlaceholder

7.1 Presentation

JSONPlaceholder est une API REST gratuite pour les tests et prototypes.

<https://jsonplaceholder.typicode.com>

7.2 Endpoints Disponibles

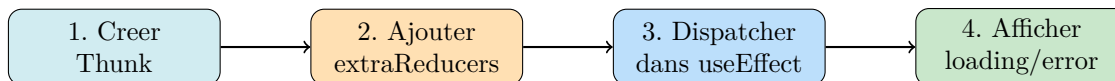
asynorange!20 Methode	Endpoint	Description
GET	/users	Liste des utilisateurs
GET	/users/1	Un utilisateur par ID
POST	/users	Creer un utilisateur
PUT	/users/1	Modifier un utilisateur
DELETE	/users/1	Supprimer un utilisateur
GET	/posts	Liste des posts
GET	/todos	Liste des todos

7.3 Exemple de Reponse

```
1 // GET /users/1
2 {
3   "id": 1,
4   "name": "Leanne Graham",
5   "username": "Bret",
6   "email": "Sincere@april.biz",
7   "phone": "1-770-736-8031 x56442",
8   "website": "hildegard.org"
9 }
10
```

8 Recapitulatif

8.1 Workflow Complet



8.2 Tableau Recapitulatif

Concept	Description
<code>createAsyncThunk</code>	Cree un thunk qui genere 3 actions (pending/-fulfilled/rejected)
<code>extraReducers</code>	Permet au slice de reagir aux actions async
<code>pending</code>	Requete en cours (loading = true)
<code>fulfilled</code>	Requete reussie (data dans payload)
<code>rejected</code>	Requete echouee (error dans payload)
<code>rejectWithValue</code>	Personnaliser le payload d'erreur

8.3 Checklist Session 17

Comprendre pourquoi les reducers ne peuvent pas etre async

Creer un thunk avec `createAsyncThunk`

Gerer les 3 etats dans `extraReducers`

Utiliser `rejectWithValue` pour les erreurs

Dispatcher le thunk dans `useEffect`

Afficher loading/error dans l'UI

Prochaine session (S18) : Formulaires & Listes avec Redux

Nous verrons comment creer des formulaires CRUD complets avec Redux.